

ROBUST SERVICES

WEEK 2, DAY 4

Idempotency and timeouts. Making the contract survive retries, and survive its own blocking code.





PART ONE: IDEMPOTENCY

Why retrying a POST creates two of everything

THE DEFINITION, IN ONE SENTENCE

EVERYTHING ELSE IS DETAIL

CALL IT ONCE, OR A HUNDRED TIMES — THE SYSTEM ENDS UP IN THE SAME STATE.

NOT THE SAME AS "SAME RESPONSE"

DELETE once returns 204. Again on the same id returns 404
— different response, same end state: still gone.

THE DESTINATION, NOT THE TRIP

It says nothing about being fast or side-effect free along the way. Only that the end state stops moving.

HOW ONE REQUEST BECOMES TWO FIRMS

THE FAILURE, STEP BY STEP

- | | | |
|---|----------------------------------|--|
| 1 | Client sends POST /firms | The server receives it and creates the firm |
| 2 | The response is lost | Connection drops. The client sees only a timeout |
| 3 | Client retries, same body | The correct instinct — and the dangerous one |
| 4 | Two identical firms | Ids 6 and 7. Same name, same figures, both real |

A server cannot tell "this never arrived" from "this worked and the reply got lost".

WHICH METHODS ARE IDEMPOTENT?

LEARN THIS TABLE AND THE RULE FOLLOWS

Method	Idempotent?	Why
GET	Yes	Reading never changes state. Call it a thousand times, FIRMS is untouched.
PUT	Yes	Same body, same id, every time. The end state does not move on the second call.
DELETE	Yes*	First call removes it. Every call after is a 404 — same end state: still gone.
POST	No	"Create" means "make a new one". The same body twice is, by design, two things.

Safe to retry and idempotent are the same question, asked from the two ends of the wire.

THE FIX: AN IDEMPOTENCY KEY

THREE STEPS, AND YOU BUILD ALL THREE NEXT

- | | | |
|----------|--------------------------------------|---|
| 1 | The client attaches a key | A header it generated itself, before the first attempt |
| 2 | The server remembers it | First time it sees a key: do the work, store the result |
| 3 | A repeat returns the original | Same key again: hand back the firm already created |

The client generates the key, not the server — a retry only works if the same key comes back.



PART TWO: TIMEOUTS

Server side. Not the call you make, the one you answer

TWO SEATS, ONE WIRE

TODAY IS THE MIRROR OF THURSDAY

THURSDAY — CALLING SOMETHING SLOW

Your client, someone else's endpoint. You choose the timeout, because you are the one waiting.

TODAY — BEING SOMETHING SLOW

Your service, someone else's client. You do not choose their patience, only your own behaviour.

Every service in a chain is somebody's dependency — including yours.

ONE BLOCKING CALL FREEZES EVERYTHING

FOUR LINES IS ALL IT TAKES

YOU WRITE THIS

```
@app.get("/slow-bad")
async def slow_bad():
    time.sleep(5)
    return {"done": True}
```

AND EVERY OTHER REQUEST PAYS FOR IT

- One event loop, one thread — every async def route shares it
- time.sleep is not awaited, so nothing yields the thread back
- /health, /firms, everything queues behind it for five seconds

Nothing crashes. Nothing logs an error. That is what makes it easy to miss.

DO NOT BLOCK THE LOOP

TWO ROUTES OUT, DEPENDING ON THE ROUTE

NOT AWAITING ANYTHING?

```
def slow_good():
```

Use a plain def. FastAPI gives it its own thread automatically.
Nothing to configure.

NEED ASYNC, BUT THE WORK BLOCKS?

```
await run_in_threadpool(...)
```

Hand it to a thread explicitly, rather than running it on the event loop.

If a route does not await anything, it should not be async def.